Guía de permisos en Windows Server: ACL/ACE, NTFS, Compartición SMB y VSS

1. Conceptos fundamentales

1.1 ACL (Access Control List)

Una **ACL** es la **lista de control de acceso** asociada a un objeto (fichero o carpeta) en NTFS. Contiene **entradas** (**ACE**) que definen qué identidades (usuarios o grupos) tienen qué permisos sobre el objeto. Hay dos tipos:

- **DACL** (Discretionary ACL): define **quién puede** hacer **qué** (permisos de acceso).
- **SACL** (System ACL): define **qué eventos** se **auditan** (lectura/escritura/éxitos/fallos).

1.2 ACE (Access Control Entry)

Una **ACE** es una **entrada individual** dentro de una ACL. Es la línea que declara **identidad** (usuario/grupo), **tipo** (Allow/Deny) y **permisos** concretos (p. ej., Read, Write, Delete).

1.3 Reglas generales sobre el cálculo de permisos efectivos

- **Acumulación**: los permisos Allow de usuario + grupos **se suman**.
- **Prioridad de Deny**: **Deny** explícito **prevalece** sobre Allow.
- **Explícito > Heredado**: una ACE establecida directamente en el objeto (explícita) prevalece sobre las heredadas.
- En caso de conflicto entre permisos, **el permiso efectivo es el más restrictivo de ellos.**

2. Permisos NTFS

Nota: se listan los **permisos estándar** (combinaciones de avanzados). Se indica si aplica a **Fichero (F)**, **Carpeta (C)** o **Ambos (F/C)**, y su **efecto** específico.

2.1 Read (Leer) — F/C

- **Fichero**: ver y abrir el contenido del fichero; leer atributos y permisos.
- **Carpeta**: ver nombres de elementos (si también se tiene *List folder*), leer atributos; **no** permite crear ni modificar.
- **Uso típico**: bibliotecas de consulta, manuales, instaladores de solo lectura.

2.2 Read & Execute (Leer y ejecutar) — F/C

- **Fichero**: todo lo que permite *Read* + **ejecutar** ejecutables/scripts.
- **Carpeta**: permite **recorrer** la carpeta y ejecutar ficheros dentro (si se tiene acceso a ellos).
- **Uso típico**: repositorios de software donde los usuarios **ejecutan** instaladores pero **no** modifican nada.

2.3 List Folder Contents (Mostrar contenido de carpeta) — C

- **Fichero**: N/A.
- **Carpeta**: **listar** los elementos de la carpeta (nombres de subcarpetas/Ficheros) y navegar por su estructura.
- **Uso típico**: permitir **navegación** por la jerarquía de carpetas sin abrir ficheros (se combina con Read/Read&Execute según necesidad).

2.4 Write (Escribir) — F/C

- **Fichero**: **crear** y **modificar** contenido (escritura), **sin** permiso para **eliminar**.
- **Carpeta**: **crear** ficheros y **subcarpetas**, **modificar** metadatos; **no** permite eliminar elementos existentes.
- **Uso típico**: "buzón de entrega" (drop folder), formularios o cargas donde se **sube** contenido pero no se elimina.

2.5 Modify (Modificar) — F/C

- **Fichero**: Read + Write + **Delete** (eliminar). **No** permite cambiar la ACL ni tomar posesión.
- **Carpeta**: permite **crear/modificar/eliminar** ficheros y subcarpetas dentro.
- **Uso típico**: áreas de **trabajo colaborativo** (departamentos/proyectos) donde se necesita crear, editar y eliminar ficheros.

2.6 Full Control (Control total) — F/C

- **Fichero**: todo lo anterior + **cambiar permisos (Change permissions)** y **tomar posesión (Take ownership)**.
- **Carpeta**: además, gestionar herencia y ACL de toda la jerarquía (según se propague).
- **Uso típico**: **administradores** de datos, cuentas de servicio de backup o propietarios delegados.

Recomendación: si **no se quiere** que un usuario pueda **eliminar**, evita asignar *Modify* y combina *Read & Execute + Write*. Si se necesita navegación por la estructura pero no apertura de ficheros, usa *List folder* sin *Read data* (mediante permisos avanzados).

2.7 Permisos NTFS avanzados

Leyenda: **F** = Fichero, **C** = Carpeta, **F/C** = ambos.

1. **Traverse folder / execute file — F/C**
    - **Fichero (F):** permite **ejecutar** el fichero (EXE, BAT, etc.).
    - **Carpeta (C):** permite **recorrer** la carpeta aunque no se pueda listar su contenido (útil para rutas profundas).
2. **List folder / read data — F/C**
    - **Fichero (F):** permite **leer datos** del fichero (abrir en modo lectura).
    - **Carpeta (C):** permite **listar** nombres de elementos dentro de la carpeta.
3. **Read attributes — F/C**
    - Leer **atributos básicos** (solo lectura, oculto, etc.). No afecta a datos.
4. **Read extended attributes — F/C**
    - Leer **atributos extendidos** definidos por aplicaciones (metadatos).
5. **Create files / write data — F/C**
    - **Fichero (F):** permite **escribir** datos en el fichero.
    - **Carpeta (C):** permite **crear ficheros** dentro de la carpeta.
6. **Create folders / append data — F/C**
    - **Fichero (F):** permite **añadir** datos al final (sin sobrescribir).
    - **Carpeta (C):** permite **crear subcarpetas**.
7. **Write attributes — F/C**
    - Cambiar **atributos básicos** (solo lectura, oculto, etc.).

8. **Write extended attributes — F/C**

    - Cambiar **atributos extendidos** (metadatos específicos de apps).

9. **Delete subfolders and files — C**

    - Permite **eliminar** elementos contenidos **aunque** el elemento no tenga su propio *Delete*. Muy relevante en jerarquías.

10. **Delete — F/C**

    - Permite **eliminar** el objeto (fichero o carpeta).

    - Nota: si se **deniega** *Delete* en el objeto pero se **concede** *Delete subfolders and files* en el contenedor, el elemento puede **seguir eliminándose** por herencia del contenedor.

11. **Read permissions — F/C**

    - Permite **ver** la ACL (permisos) del objeto.

12. **Change permissions — F/C**

    - Permite **modificar** la ACL (DACL) del objeto.

13. **Take ownership — F/C**

    - Permite **asumir la propiedad** del objeto (el propietario siempre puede modificar permisos).

Consejos prácticos

- Un "**drop folder**" se construye combinando en la carpeta **Create files / write data** y **deny List folder / read data**, de modo que los usuarios **suben** pero **no ven**.

- Para permitir **navegar** sin listar, concede **Traverse folder** y deniega **List folder**.

2.8 Correspondencia entre permisos estándar y avanzados

Los permisos estándar son “paquetes” de avanzados. La siguiente lista refleja la composición más utilizada (puede variar levemente por versión/GUI).

2.8.1 En Ficheros (F)

- **Read (Leer)**
  Incluye: *Read data*, *Read attributes*, *Read extended attributes*, *Read permissions*.

- **Read & Execute (Leer y ejecutar)**
  Read + *Execute file*.

- **Write (Escribir)**
  Write data, *Append data*, *Write attributes*, *Write extended attributes*, *Read permissions*.

- **Modify (Modificar)**
  Read & Execute + *Write* + *Delete*.

- **Full Control (Control total)**
  Modify + *Change permissions* + *Take ownership*.

2.8.2 En carpetas (C)

- **Read (Leer)**
  List folder, *Read attributes*, *Read extended attributes*, *Read permissions*.

- **Read & Execute (Leer y ejecutar)**
  Read + *Traverse folder*.

- **List folder contents (Mostrar contenido)**
  Prácticamente igual que *Read & Execute* pero aplicado al **nivel de carpeta** (listado/navegación).

- **Write (Escribir)**
  Create files / write data, *Create folders / append data*, *Write attributes*, *Write extended attributes*, *Read permissions*.

- **Modify (Modificar)**
  Read & Execute + *Write* + *Delete*

- **Full Control (Control total)**
  Modify + *Change permissions* + *Take ownership* (administración completa de la jerarquía según herencia).

3. Permisos de compartición (SMB) — Read, Change, Full Control

Se aplican al **recurso compartido** (\\Servidor\Recurso) y solo afectan al **acceso por red**.

- **Read**: listar, abrir y ejecutar en **solo lectura**.
- **Change**: **Read** + crear/modificar/**eliminar** ficheros y carpetas.
- **Full Control**: **Change** + cambiar **permisos del recurso compartido** (no los NTFS).

Relación NTFS ↔ Compartición
El **permiso efectivo** para cada operación es el **más restrictivo** entre **NTFS** y **Compartición**.

- Ej.: Share = Change, NTFS = Read → **solo lectura**.
- Ej.: Share = Read, NTFS = Modify → **solo lectura**.

4. Buenas prácticas en entornos AD + Windows Server

4.1 Modelo de grupos (AGDLP / AGUDLP)

- **Accounts** → **Global groups** (rol/área) → **Domain Local groups** (recurso) → **Permissions** en la carpeta.
- Asignar permisos **siempre a grupos**, **nunca** a usuarios individuales. Facilita auditoría y mantenimiento.

4.2 Dónde controlar realmente

- Estilo clásico: **Compartición "amplia"** (p. ej., *Authenticated Users = Change/Full*) y **granularidad en NTFS**.
- Estilo conservador: Compartición = **Change** para el/los grupo/s de acceso; NTFS define el detalle.
- En ambos casos, **prevalece el más restrictivo**.

4.3 Herencia y diseño

- Se deben diseñar ACL **limpias** en un nivel alto (p. ej., `D:\Shares\Departamento`) y aprovechar la **herencia**.

- Se debe deshabilitar la herencia **sólo** cuando sea necesario. Se debe evitar el uso de **Deny** salvo en casos específicos.

4.4 Access-Based Enumeration (ABE)

- Se debe habilitar en el share para que los usuarios **no vean** carpetas a las que no tienen permisos. Mejora **privacidad** y **usabilidad**.

4.5 Administración y seguridad

- Se deben separar funciones: **Administradores de datos**, **Backup Operators**, **Service Accounts** con **mínimo privilegio**.

- Se deben **auditar** los accesos sensibles (SACL) y usa **VSS**/copias de seguridad con pruebas de **restauración** periódicas.

- Se deben **documentar** los grupos, propietarios y justificar los permisos asignados.

5. Ejemplos de aplicación

1. **Departamento colaborativo**
    - **Compartición**: grupo del departamento = **Change**.
    - **NTFS**: grupo = **Modify**, otros = **Read** (si procede).
    - **Efecto**: edición completa para el departamento; lectura para terceros.
2. **Home folders**
    - **Compartición**: usuarios = **Change**, ABE **ON**.
    - **NTFS**: cada usuario = **Full Control** en su carpeta; **Admins/Backup** con permisos administrativos.
    - **Efecto**: acceso exclusivo del usuario a su carpeta.

3. **Drop folder**
    - **Compartición**: **Change**.
    - **NTFS**: **Write** permitido, **List/Read** denegado para usuarios.
    - **Efecto**: pueden **subir** ficheros, **no** ver ni borrar los de otros.

6. VSS (Volume Shadow Copy Service)

6.1 ¿Qué es?

VSS (Volume Shadow Copy Service) es el servicio de Windows que crea **instantáneas** puntuales de un volumen NTFS. Permite recuperar **versiones anteriores** de ficheros/carpetas sin recurrir a una restauración completa desde el backup.

6.2 Utilidad

- **Recuperación rápida** ante borrados o sobrescrituras accidentales ("Versiones anteriores").
- **Reducción de carga** sobre el equipo de TI: muchos casos se resuelven sin recurrir a una restauración desde el backup.
- **Complemento**, no sustituye a los **backups** tradicionales (no protege ante pérdida del volumen entero si reside en el mismo disco).

6.3 Cómo se activa (Windows Server)

1. En el servidor, clic derecho en el volumen (p. ej., **D:**) → **Properties** → pestaña **Shadow Copies**.
2. **Habilitar** en el volumen de datos.
3. Configurar:
    - **Programación** (frecuencia de snapshots).
    - **Límite de espacio** (cuota para almacenar copias).

4. Los clientes pueden ver **versiones anteriores** en Propiedades de ficheros/carpetas (si el volumen compartido tiene VSS).

   También se puede orquestar con **Task Scheduler**/PowerShell o soluciones de backup que usan VSS para coherencia de aplicaciones.

6.4 Implicaciones, ventajas e inconvenientes

Ventajas

- **RTO muy bajo** para recuperar ficheros y carpetas (segundos/minutos).
- Facilita la **auto-recuperación** por parte del usuario (si se expone vía share).
- Integrado en Windows; compatible con aplicaciones vía **VSS writers**.

Inconvenientes / Consideraciones

- **No sustituye** al backup: si el volumen se corrompe o el servidor se pierde, las *Shadow Copies* pueden perderse.
- **Espacio**: requiere **cuota**; cuando se agota, VSS **purga** copias antiguas.
- **Rendimiento**: muchas escrituras sobre el volumen o volúmenes muy activos pueden aumentar la **fragmentación** y la **sobrecarga** de I/O.
- **Retención limitada**: la "historia" depende del **espacio** y de la **frecuencia** configurada.
- **Seguridad**: las copias heredan ACL; los usuarios con permisos pueden recuperar sus propios ficheros, lo cual es deseable pero debe **alinearse** con políticas de seguridad definidas en la organización.

Buenas prácticas con VSS

- Ubica VSS en volúmenes de **datos** (no solo sistema) y asigna una **cuota** suficiente (10–20% como referencia inicial; ajusta según uso).
- Define una **programación** que equilibre puntos de recuperación y espacio (por ejemplo, varias veces al día en horarios laborales).
- Mantén **backups externos** (cinta/nube/otro repositorio): VSS es una **capa adicional** de recuperación rápida.

- Monitoriza **espacio VSS** y **eventos**; documenta el **procedimiento de restauración** para usuarios y helpdesk.

7. Resumen

- **ACL/ACE**: la ACL es la lista; la ACE es la entrada que otorga/deniega permisos a un recurso. **Deny** prevalece y lo **explícito** prevalece sobre lo heredado.

- **NTFS (más → menos restrictivo)**: Read → Read & Execute → List folder (carpetas) → Write → Modify → Full Control. Conoce el **efecto** por **fichero/carpeta**.

- **Compartición (SMB)**: Read, Change, Full Control. El **permiso efectivo** es el **más restrictivo** entre NTFS y share.

- **Buenas prácticas**: **AGDLP**, permisos a **grupos**, herencia bien diseñada, **ABE** habilitado, mínima dependencia de **Deny**, y documentación.

- **VSS**: habilita **versiones anteriores** para recuperación rápida; se debe configurar el **espacio** y **programación**; úsalo junto con **backups** formales.